Tutorial: Create Logical Data Warehouse with serverless SQL pool

Article • 03/29/2023

In this tutorial, you will learn how to create a Logical Data Warehouse (LDW) on top of Azure storage and Azure Cosmos DB.

LDW is a relational layer built on top of Azure data sources such as Azure Data Lake storage (ADLS), Azure Cosmos DB analytical storage, or Azure Blob storage.

Create an LDW database

You need to create a custom database where you will store your external tables and views that are referencing external data sources.

SQL

```
CREATE DATABASE Ldw
      COLLATE Latin1_General_100_BIN2_UTF8;
```

This collation will provide the optimal performance while reading Parquet and Azure Cosmos DB. If you don't want to specify the database collation, make sure that you specify this collation in the column definition.

Configure data sources and formats

As a first step, you need to configure data source and specify file format of remotely stored data.

Create data source

Data sources represent connection string information that describes where your data is placed and how to authenticate to your data source.

One example of data source definition that references public ECDC COVID 19 Azure Open Data Set is shown in the following example:

SQL

```
CREATE EXTERNAL DATA SOURCE ecdc_cases WITH (
    LOCATION =
'https://pandemicdatalake.blob.core.windows.net/public/curated/covid-
19/ecdc_cases/'
);
```

A caller may access data source without credential if an owner of data source allowed anonymous access or give explicit access to Microsoft Entra identity of the caller.

You can explicitly define a custom credential that will be used while accessing data on external data source.

- Managed Identity of the Synapse workspace
- Shared Access Signature of the Azure storage
- Custom Service Principal Name or Azure Application identity.
- Read-only Azure Cosmos DB account key that enables you to read Azure Cosmos DB analytical storage.

As a prerequisite, you will need to create a master key in the database:

SQL

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'Setup you password - you need to
create master key only once';
```

In the following external data source, Synapse SQL pool should use a managed identity of the workspace to access data in the storage.

SQL

```
CREATE DATABASE SCOPED CREDENTIAL WorkspaceIdentity
WITH IDENTITY = 'Managed Identity';
GO
CREATE EXTERNAL DATA SOURCE ecdc_cases WITH (
    LOCATION =
'https://pandemicdatalake.blob.core.windows.net/public/curated/covid-
19/ecdc_cases/',
    CREDENTIAL = WorkspaceIdentity
);
```

In order to access Azure Cosmos DB analytical storage, you need to define a credential containing a read-only Azure Cosmos DB account key.

SQL

```
CREATE DATABASE SCOPED CREDENTIAL MyCosmosDbAccountCredential
WITH IDENTITY = 'SHARED ACCESS SIGNATURE',
     SECRET =
's5zarR2pT0JWH9k8roipnWxUYBegOuFGjJpSjGlR36y86cW0GQ6RaaG8kGjsRAQoWMw1QKTkkX8HQt
FpJjC8Hg==';
```

Any user with the Synapse Administrator role can use these credentials to access Azure Data Lake storage or Azure Cosmos DB analytical storage. If you have low privileged users that do not have Synapse Administrator role, you would need to give them an explicit permission to reference these database scoped credentials:

SQL

```
GRANT REFERENCES ON DATABASE SCOPED CREDENTIAL::WorkspaceIdentity TO <user>
GO
GRANT REFERENCES ON DATABASE SCOPED CREDENTIAL::MyCosmosDbAccountCredential TO
<user>
GO
```

Find more details in grant DATABASE SCOPED CREDENTIAL permissions page.

Define external file formats

External file formats define the structure of the files stored on external data source. You can define Parquet and CSV external file formats:

SQL

```
CREATE EXTERNAL FILE FORMAT ParquetFormat WITH (  FORMAT_TYPE = PARQUET );
GO
CREATE EXTERNAL FILE FORMAT CsvFormat WITH (  FORMAT_TYPE = DELIMITEDTEXT );
```

For more information, see Use external tables with Synapse SQL and CREATE EXTERNAL FILE FORMAT to describe format of CSV or Parquet files.

Explore your data

Once you set up your data sources, you can use the `OPENROWSET` function to explore your data. The OPENROWSET function reads content of a remote data source (for example file) and returns the content as a set of rows.

SQL

```
select top 10  *
from openrowset(bulk 'latest/ecdc_cases.parquet',
                data_source = 'ecdc_cases',
                format='parquet') as a
```

The `OPENROWSET` function will give you information about the columns in the external files or containers and enable you to define a schema of your external tables and views.

Create external tables on Azure storage

Once you discover the schema, you can create external tables and views on top of your external data sources. The good practice is to organize your tables and views in databases schemas. In the following query you can create a schema where you will place all objects that are accessing ECDC COVID data set in Azure data Lake storage:

SQL

```
create schema ecdc_adls;
```

The database schemas are useful for grouping the objects and defining permissions per schema.

Once you define the schemas, you can create external tables that are referencing the files. The following external table is referencing the ECDC COVID parquet file placed in the Azure storage:

SQL

```
create external table ecdc_adls.cases (
    date_rep                  date,
    day                       smallint,
    month                     smallint,
    year                      smallint,
    cases                     smallint,
    deaths                    smallint,
    countries_and_territories  varchar(256),
```

```
    geo_id                      varchar(60),
    country_territory_code      varchar(16),
    pop_data_2018               int,
    continent_exp               varchar(32),
    load_date                   datetime2(7),
    iso_country                 varchar(16)
) with (
    data_source= ecdc_cases,
    location = 'latest/ecdc_cases.parquet',
    file_format = ParquetFormat
);
```

Make sure that you use the smallest possible types for string and number columns to optimize performance of your queries.

Create views on Azure Cosmos DB

As an alternative to external tables, you can create views on top of your external data.

Similar to the tables shown in the previous example, you should place the views in separate schemas:

SQL

```
create schema ecdc_cosmosdb;
```

Now you are able to create a view in the schema that is referencing an Azure Cosmos DB container:

SQL

```
CREATE OR ALTER VIEW ecdc_cosmosdb.Ecdc
AS SELECT *
FROM OPENROWSET(
      PROVIDER = 'CosmosDB',
      CONNECTION = 'Account=synapselink-cosmosdb-sqlsample;Database=covid',
      OBJECT = 'Ecdc',
      CREDENTIAL = 'MyCosmosDbAccountCredential'
    ) WITH
     ( date_rep varchar(20),
       cases bigint,
       geo_id varchar(6)
     ) as rows
```

To optimize performance, you should use the smallest possible types in the `WITH` schema definition.

> **Note**
>
> You should place your Azure Cosmos DB account key in a separate credential and reference this credential from the `OPENROWSET` function. Do not keep your account key in the view definition.

Access and permissions

As a final step, you should create database users that should be able to access your LDW, and give them permissions to select data from the external tables and views. In the following script you can see how to add a new user that will be authenticated using Microsoft Entra identity:

SQL

```
CREATE USER [jovan@contoso.com] FROM EXTERNAL PROVIDER;
GO
```

Instead of Microsoft Entra principals, you can create SQL principals that authenticate with the login name and password.

SQL

```
CREATE LOGIN [jovan] WITH PASSWORD = 'My Very strong Password ! 1234';
CREATE USER [jovan] FROM LOGIN [jovan];
```

In both cases, you can assign permissions to the users.

SQL

```
DENY ADMINISTER DATABASE BULK OPERATIONS TO [jovan@contoso.com]
GO
GRANT SELECT ON SCHEMA::ecdc_adls TO [jovan@contoso.com]
GO
GRANT SELECT ON OBJECT::ecdc_cosmosDB.cases TO [jovan@contoso.com]
GO
GRANT REFERENCES ON DATABASE SCOPED CREDENTIAL::MyCosmosDbAccountCredential TO
```

```
[jovan@contoso.com]
GO
```

The security rules depend on your security policies. Some generic guidelines are:

- You should deny `ADMINISTER DATABASE BULK OPERATIONS` permission to the new users because they should be able to read data only using the external tables and views that you prepared.
- You should provide `SELECT` permission only to the tables that some user should be able to use.
- If you are providing access to data using the views, you should grant `REFERENCES` permission to the credential that will be used to access external data source.

This user has minimal permissions needed to query external data. If you want to create a power-user who can set up permissions, external tables and views, you can give `CONTROL` permission to the user:

SQL

```
GRANT CONTROL TO [jovan@contoso.com]
```

Role-based security

Instead of assigning permissions to the individual uses, a good practice it to organize the users into roles and manage permission at role-level. The following code sample creates a new role representing the people who can analyze COVID-19 cases, and adds three users to this role:

SQL

```
CREATE ROLE CovidAnalyst;

ALTER ROLE CovidAnalyst ADD MEMBER [jovan@contoso.com];
ALTER ROLE CovidAnalyst ADD MEMBER [milan@contoso.com];
ALTER ROLE CovidAnalyst ADD MEMBER [petar@contoso.com];
```

You can assign the permissions to all users that belong to the group:

SQL

```
GRANT SELECT ON SCHEMA::ecdc_cosmosdb TO [CovidAnalyst];
GO
DENY SELECT ON SCHEMA::ecdc_adls TO [CovidAnalyst];
GO
DENY ADMINISTER DATABASE BULK OPERATIONS TO [CovidAnalyst];
```

This role-based security access control might simplify management of your security rules.

Next steps

- To learn how to connect serverless SQL pool to Power BI Desktop and create reports, see Connect serverless SQL pool to Power BI Desktop and create reports.
- To learn how to use External tables in serverless SQL pool see Use external tables with Synapse SQL